TransportFlow-Transportation-Optimization



Student Name	Student ID	Student Name	Student ID
مينا أمجد بولس جرجس	4241053	ميران ناجح ابراهيم هرمينا	4241081
إسلام محمد على حسن	4241055	محمد حجاجى عبدالرحمن احمد حسين	4241085
دميانه جابر داود عياد يونان	4241058	محمد حازم بكر ابوالوفا عبدالله	4241086
فيكتوريا هانى رزق فرج	4241059	علي سيد علي عبدالمالك	4241090
عبدالرحمن محمود حسن محمود حسن	4241060	مكاريوس رومانى خيرى بشرى باسليوس	4241092
اميره عصام سعد عطية	4241061	منصور خالد منصور محمد	4241095
ايمن كميل ثابت مسعد مقار	4241064	مريت سعد شفيق فرج	4241096
رغد عبدالناصر احمد عبدالمجيد	4241068	خالد محمد عبد الرحمن عبد الحميد	4241099
عمرو محمد عبد المعز احمد	4241075	—	—

Transportation Model in Operations Research

1. Definition

The Transportation Model is a mathematical model used in Operations Research to determine the most cost-effective way to transport goods from several sources (suppliers) to several destinations (consumers). The goal is to minimize the total transportation cost while meeting supply and demand constraints.

2. Example

Suppose there are four companies: Company 1, Company 2, Company 3, and Company 4. They supply employees (units) to ten depots: Bank, Hospital, Store, Telecom, University, Insurance, Airline, Factory, Ministry, and Media. Each route (e.g., from Company 1 to Bank) has a specific transportation cost per unit. Each company has a fixed supply capacity, and each depot has a fixed demand. Three demand scenarios are studied: demand greater than supply, supply greater than demand, and balanced supply and demand.

3. Objective

The objective is to determine how many units should be transported from each company to each depot so that all demands are satisfied, supplies are not exceeded, and the total cost is minimized.

4. Mathematical Formulation

Let x_{ij} be the number of units transported from source i to destination j, and c_{ij} be the cost per unit of transportation. The objective function is:

- Minimize $Z = \Sigma\Sigma\, c_{ij} * x_{ij}$
- Subject to:
- $\Sigma\, x_{ij}$ (over j) = $Supply_i$ for all i
- $\Sigma\, x_{ij}$ (over i) = $Demand_j$ for all j
- $x_{ij} \geq 0$

These equality constraints hold when total supply equals total demand. If they are not equal, the problem is first balanced by adding a dummy source (when demand is greater than supply) or a dummy depot (when supply is greater than demand) with zero transportation cost.

5. Methods of Solution

The solution of a transportation problem generally involves three stages:

Stage 1: Finding an Initial Feasible Solution

Common methods used to find an initial feasible solution include:

- Least Cost Method (LCM)
- Vogel's Approximation Method (VAM)

These methods provide a starting solution that satisfies all supply and demand constraints but may not be optimal. In this project, the Least Cost Method and VAM are both implemented and compared, and the VAM solution is used as the starting point for the MODI optimization.

Stage 2: Optimality Test

After obtaining an initial feasible solution, optimality is tested using the u–v (Potentials) Method or the MODI Method. If all opportunity costs (Δ) are greater than or equal to zero, the solution is optimal. Otherwise, further improvement is required.

Stage 3: Improvement (Optimization Step)

If the solution is not optimal, the next step is to improve it using an iterative process:

- • Select the cell with the most negative Δ value.
- • Construct a closed loop.
- • Adjust the allocations along the loop alternately (+ and –).
- • Repeat until all Δ ≥ 0 (optimal solution).

6. Code Explanation

The code is divided into logical sections that represent the solution process of the Transportation Problem. In the notebook, each section is placed in its own cell.

6.1 Importing Libraries

```
import numpy as np
import matplotlib.pyplot as plt
```

Explanation:

NumPy is used for numerical operations and Matplotlib for visualization.

6.2 Defining Input Data

```
cost = np.array([
    [1,  3,  9,  8,  5,  2,  2,  4, 12,  3],
    [4,  4, 14, 12,  9,  2,  7,  7, 16,  2],
    [18, 20, 8, 10, 13, 19, 20, 22, 4, 21],
    [13, 15, 4,  6,  8, 15, 16, 18, 1, 17]
], dtype=np.float64)

cities = ["Company 1", "Company 2", "Company 3", "Company 4"]
depots = ["Bank", "Hospital", "Store", "Telecom", "University", "Insurance",
          "Airline", "Factory", "Ministry", "Media"]
```

```
supply = [40, 60, 50, 50]
```

Explanation:

cost: a 4 × 10 matrix showing the unit transportation cost (EGP) between every company (row) and every depot (column).

cities and depots: the names of the sources (Company 1, Company 2, Company 3, Company 4) and the destinations (Bank, Hospital, Store, Telecom, University, Insurance, Airline, Factory, Ministry, Media).

supply: the available employees in each company. The total supply is 200 employees.

The demand is not fixed in this section. It is defined separately for each of the three scenarios (see Section 6.9), so the same cost matrix and supply can be tested against different demand situations.

6.3 Balance Check

```
def balance_check(cost, supply, demand, cities, depots):
```

Explanation:

The function compares total supply with total demand and handles three cases:

1. Total supply = total demand: the problem is balanced and nothing is added.

2. Total demand > total supply: a dummy source (an extra row of zero costs) is added, with supply equal to the difference.

3. Total supply > total demand: a dummy demand (an extra column of zero costs) is added, with demand equal to the difference.

Because the dummy costs are zero, they do not affect the total cost. The function returns the extended cost matrix, supply, demand, and the updated names of sources and destinations.

6.4 Implementing the VAM (Vogel's Approximation Method)

```
def vogel_approximation(cost, supply, demand):
```

Steps in the code:

1. For each row and column that still has supply or demand, the two lowest costs are found, and their difference is taken as a penalty (if only one cost remains, it is used as the penalty).

2. The row or column with the highest penalty is selected first (a row is preferred if the penalties are equal).

3. The cell with the lowest cost in that row/column is chosen for allocation.

4. The minimum of the current supply and demand is allocated there.

5. The satisfied row or column is removed by setting its costs to infinity (the original costs are kept to compute the total cost).

6. Steps 1–5 repeat until all supplies and demands are zero.

Result:

The function returns the allocation matrix and the total cost of the initial feasible solution.

6.5 Implementing the Least Cost Method

```
def least_cost_method(cost, supply, demand):
```

Explanation:

At each iteration the code searches all cells that still have remaining supply and demand and selects the cell with the lowest cost. It allocates the minimum of the remaining supply and demand to that cell, removes the satisfied row or column, and repeats until everything is allocated. This second initial solution is used to compare against VAM.

6.6 Applying the MODI Method (Optimization Step)

```
def complete_basis(allocation, cost):
def potentials(cost, basis):
def loop_path(basis, row, col):
def modi(cost, start_allocation):
```

Explanation:

complete_basis: makes sure the solution has m + n − 1 basic cells. If the initial solution is degenerate, zero-allocation cells with the lowest costs are added, using a union-find structure so that no closed loop is created.

potentials: calculates the u and v potentials for each row and column from the basic cells, starting with u = 0 for the first row.

Then modi computes the reduced costs (Δ) for the non-basic cells using the formula:

Δ_{ij} = c_{ij} - (u_i + v_j)

If no Δ is negative, the current solution is optimal. Otherwise, the cell with the most negative Δ enters the basis, and loop_path finds the closed loop of basic cells through that cell.

The loop cells are marked alternately (+) and (−). The value θ is the smallest allocation among the (−) cells; it is added to the (+) cells and subtracted from the (−) cells, and the (−) cell that reaches zero leaves the basis. Each iteration is recorded (entering cell, Δ, θ, and the cost before and after), and the process repeats until all Δ ≥ 0.

In the code, MODI always starts from the VAM allocation and returns the optimal allocation, the optimal total cost, and the list of improvement steps.

6.7 Printing and Solving One Scenario

```
def print_allocation(allocation, rows, cols):
def solve_case(title, cost, supply, demand, cities, depots):
```

Explanation:

print_allocation prints every route with a positive allocation (for example, Company 1 -> Bank = 20 units).

solve_case solves one complete scenario in this order: balance check, VAM, Least Cost Method, comparison of the two initial solutions, and then MODI starting from the VAM solution. It prints the total cost of each method and the number of MODI improvement steps, and returns the results so they can be plotted and compared.

6.8 Visualization

```
def plot_case(result):
```

Explanation:

For each scenario, the code draws three charts:

1. A bar chart comparing the total cost of VAM, Least Cost, and MODI.

2. A bar chart showing the transportation cost of each route in the optimal solution (allocation × unit cost).

3. A pie chart showing each company's share of the shipped employees (dummy rows and columns are excluded).

6.9 Running the Three Scenarios

```
def run_three_scenarios():
```

Explanation:

The function defines three demand vectors, solves each scenario with solve_case, plots the charts, prints a final cost comparison, and draws a bar chart of the optimal MODI cost of the three scenarios. The total supply is 200 employees in all scenarios:

Scenario	Demand per depot	Total demand	Balance case
1	Bank = 30, Hospital = 30, all other depots = 20	220	Demand > Supply (dummy source of 20 added)

Scenario	Demand per depot	Total demand	Balance case
2	Bank = 15, Hospital = 15, Media = 10, all other depots = 20	180	Supply > Demand (dummy demand of 20 added)
3	All depots = 20	200	Balanced (no dummy needed)

---

7. Results and Discussion

7.1 Results Summary

Companies (Sources): Company 1, Company 2, Company 3, Company 4

Depots (Destinations): Bank, Hospital, Store, Telecom, University, Insurance, Airline, Factory, Ministry, Media

Total supply: 200 employees

Total cost of each method in the three scenarios:

Scenario	VAM (EGP)	Least Cost (EGP)	MODI – Optimal (EGP)	MODI steps	Better initial solution
1: Demand > Supply	1310	1260	1060	5	Least Cost
2: Supply > Demand	835	1235	780	2	VAM
3: Balanced	1150	1280	1100	6	VAM

7.2 Final Allocations (MODI)

Scenario 1 – Demand greater than Supply (220 vs 200):

Company 1 → Bank = 20 units
Company 1 → Airline = 20 units
Company 2 → Hospital = 20 units
Company 2 → Insurance = 20 units
Company 2 → Media = 20 units

Company 3 → Store = 10 units
Company 3 → Telecom = 20 units
Company 3 → Ministry = 20 units
Company 4 → Bank = 10 units
Company 4 → Hospital = 10 units
Company 4 → Store = 10 units
Company 4 → University = 20 units
Dummy Source → Factory = 20 units (unmet demand)
Total Transportation Cost: 1060 EGP

Scenario 2 – Supply greater than Demand (200 vs 180):

Company 1 → Bank = 15 units
Company 1 → Airline = 20 units
Company 1 → Factory = 5 units
Company 2 → Hospital = 15 units
Company 2 → Insurance = 20 units
Company 2 → Factory = 15 units
Company 2 → Media = 10 units
Company 3 → Telecom = 10 units
Company 3 → Ministry = 20 units
Company 3 → Dummy Demand = 20 units (idle employees)
Company 4 → Store = 20 units
Company 4 → Telecom = 10 units
Company 4 → University = 20 units
Total Transportation Cost: 780 EGP

Scenario 3 – Balanced Supply and Demand (200 vs 200):

Company 1 → Airline = 20 units
Company 1 → Factory = 20 units
Company 2 → Hospital = 20 units
Company 2 → Insurance = 20 units
Company 2 → Media = 20 units
Company 3 → Store = 10 units
Company 3 → Telecom = 20 units
Company 3 → Ministry = 20 units
Company 4 → Bank = 20 units
Company 4 → Store = 10 units

Company 4 → University = 20 units

Total Transportation Cost: 1100 EGP

7.3 Discussion

In every scenario, MODI improved the initial VAM solution until all $\Delta \geq 0$, which means the final solutions are optimal: from 1310 EGP to 1060 EGP in Scenario 1 (5 steps), from 835 EGP to 780 EGP in Scenario 2 (2 steps), and from 1150 EGP to 1100 EGP in Scenario 3 (6 steps).

VAM was not always the better starting point. In Scenario 1 the Least Cost Method gave a better initial solution (1260 EGP against 1310 EGP for VAM), while in Scenarios 2 and 3 VAM was clearly better (835 against 1235, and 1150 against 1280). This is why the optimality test of MODI is needed whichever initial method is used.

Scenario 1 is not balanced: total demand (220) is greater than total supply (200), so a dummy source of 20 units with cost 0 was added. The 20 units sent from the dummy source are not real shipments; they represent demand that cannot be met. In the optimal solution this shortage falls entirely on Factory.

Scenario 2 is not balanced either: total supply (200) is greater than total demand (180), so a dummy demand of 20 units with cost 0 was added. The 20 units sent to the dummy demand (from Company 3) are idle employees.

Scenario 3 is balanced (200 = 200), so no dummy source or demand was needed.

The three optimal costs (1060, 780, and 1100 EGP) should not be compared directly as a ranking of the scenarios, because each scenario has a different demand distribution across the depots.

The code efficiently distributes the available supplies to satisfy the demands at minimum cost, and the results are clear and logically consistent with the problem's goal: minimizing the overall shipping cost.

The project demonstrates the power of Python in solving real-world optimization problems like logistics and transportation planning.

Conclusion

The Python implementation successfully solved the Transportation Problem in three scenarios (demand greater than supply, supply greater than demand, and balanced) using the Least Cost Method and VAM to find initial solutions and the MODI method to optimize them.

The final solutions minimized the total cost to 1060 EGP, 780 EGP, and 1100 EGP respectively, and in each case all $\Delta \geq 0$, confirming that the chosen allocations are optimal.

Unbalanced problems are handled automatically by adding a dummy source or a dummy demand with zero cost.

This approach can easily be extended to larger transportation networks or integrated with real data for business use.

8. Applications

Field	Application
Logistics	Finding the least-cost distribution of goods between warehouses and outlets.
Manufacturing	Transporting raw materials from suppliers to factories.
Distribution	Optimizing the delivery routes between warehouses and customers.
Energy	Allocating electricity from generation plants to cities.
Services	Allocating workforce or equipment among service locations.

9. Benefits

- • Minimizes total transportation cost.
- • Improves resource utilization.
- • Helps make data-driven operational decisions.
- • Enhances overall operational efficiency.

GitHub Link :

https://github.com/mohammedhazem11/TransportFlow-Transportation-Optimization/tree/main

